2. Design Specification

2.1 High-Level Architecture (Large Scale)

Lead: Main Group (Collaborative)
[Insert overall System Architecture Diagram. Explain the flow of dat

a from J1's edge devices, through J2's Kafka brokers, to J3's UI, all hosted on J4's infrastructure.]

The Disaster Response System operates on an event-driven, decoupled microservices architecture designed for real-time data ingestion and high availability.

The data lifecycle begins at the edge with J1 (Sensors & Mobile Devices), which capture raw disaster reports and telemetry data. This data is streamed via MQTT/Kafka into the J2 processing pipeline, where AI models classify the severity and parse the data before storing it in a centralized PostgreSQL database. The processed events are then pushed in real-time via WebSockets to the J3 Dashboard (Next.js), which provides situational awareness and resource management capabilities to the end-users. All of these services are routed and secured through J4's overarching infrastructure (API Gateway, Keycloak authentication, and container orchestration).

2.1.1 J1 Components: [Describe MQTT brokers, mobile data capture logic.]

J1 Components: The J1 subsystem is designed as a **distributed edge-computing architecture**, split into three primary layers: the **Sensor Layer**, the **Gateway Layer**, and the **Mobile Edge Layer**. This design ensures that disaster monitoring and reporting can continue even if the connection to the primary cloud backend is severed.

2.1.1.1 Component Breakdown

Component	Technology Stack	Primary Responsibility
Flood Detection Node	ESP32, Ultrasonic, DHT11	Measures water levels and ambient humidity; transmits via LoRa.
Landslide Detection Node	ESP32, MPU6050, Soil Moisture	Detects ground tilt and soil saturation; transmits via LoRa.

Central Gateway Node	ESP32, LoRa, WiFi	Aggregates field data, manages local buffers, and bridges to MQTT/HTTP.
Mobile Application	Flutter (Dart)	Provides UI for report submission and real-time dashboard monitoring.
Edge Storage (Mobile)	SQLite	Manages the persistent local event queue for offline report retention.
Edge Storage (IoT)	SPIFFS (Internal Flash)	Buffers sensor payloads on the Central Node during WiFi outages.

2.1.1.2 Architectural Layers

1. IoT Sensor Layer (The Field)
Consists of specialized ESP32 nodes deployed in high-risk zones.

- **Communication:** Uses **LoRa (Long Range)** protocol to transmit data to the Central Node. This ensures connectivity in geographically challenging areas where WiFi or Cellular signals are non existent.
- **Power Management:** Optimized for low-power consumption to ensure longevity during prolonged disasters.

2. Central Communication Hub (The Gateway)
Acts as the intelligent bridge between the field sensors and the cloud.

- **Data Aggregation:** Receives and parses asynchronous LoRa packets from multiple sensor nodes.
- **Dual-Path Protocol:** Primarily uses **MQTT (HiveMQ)** for real-time telemetry and **HTTP** for external weather API integration.
- **Resilience Logic:** Monitors network heartbeats. If connectivity fails, it shifts to an internal buffering state to prevent data loss.

3. Mobile Edge Layer (The User Interface)
A Flutter-based application designed for both public users and responders.

- **Offline-First Logic:** Unlike standard apps, every user report is treated as an "Event Object" and committed to a local **SQLite database** *before* any upload attempt is made.
- **Synchronization Service:** A background process that utilizes an exponential backoff strategy to sync the SQLite queue with the backend once a stable internet

connection (4G/WiFi) is detected.

- **GPS Service:** Automatically attaches precise coordinates to help requests, even when the map tiles cannot load due to offline status.

2.1.1.3 Data Flow Summary

1. **Sensors** —> **LoRa** —>**Central Node**
2. **Central Node** —> **MQTT** —> **Cloud Dashboard** (If Online)
3. **Central Node** —> **Internal Flash** (If Offline)
4. **Mobile User** —> **SQLite** —>**HTTP API** —> **Backend**

2.1.2 J2 Components: [Describe AI prediction models, Postgres DB schemas, Spark jobs.]

The J2 service is structured as a single FastAPI microservice with the following internal components:

FastAPI Application Layer

- Entry point: app/main.py, served by Uvicorn on port 8082.
- Core routes defined in app/api/routes.py; agent routes defined in app/api/agent_routes.py and registered via app.include_router.
- Exposes GET /health, GET /api/v1/intelligence, and POST /api/v1/intelligence/agent/allocate.
- FastAPI dependency injection handles database session management per request.
- Interactive API documentation automatically available at /docs on startup.

SQLAlchemy ORM Models

Models are defined in app/models/ and cover the following domain entities:

- Organisational: Division, DivisionResource
- Sensing: IoTDevice, IoTDeviceData, BaseWaterLevel, RainfallData, SoilMoisture, TemperatureData, SPIData
- Risk: FloodSeverity, DisasterRisk, ConsiderationScore
- Response: ActiveIncident, DeployableAsset, DispatchRecord, Shelter, PublicAlert, Report

All models are imported in app/models to ensure Alembic autogeneration can detect them.

Alembic Migration Engine

- Migration scripts stored in migrations/versions/.

- Alembic environment wired through app.models metadata for autogeneration support.
- Migrations run automatically on container startup via the Dockerfile entrypoint (alembic upgrade head).
- Each revision carries a descriptive message for traceability.

PostgreSQL Database

- PostgreSQL 15 or later used as the persistence layer.
- Database connection managed via the DATABASE_URL environment variable using the psycopg2 driver.
- Schema fully managed by Alembic; no ad-hoc DDL is applied outside the migration chain.

Consideration Score Pipeline

- Implemented in app/utils/consideration_score.py as the compute_consideration_scores function.
- Accepts prediction rows (with hazard class probabilities) and population rows as inputs.
- Normalises population values between 0 and 1, identifies the dominant hazard class, and applies severity multipliers: Normal 0.10, Moderate 0.40, Severe 0.75, Extreme 1.00.
- Applies logistic scaling with a configurable constant k (default 10.0) to bound all output scores between 0 and 1.
- Returns a sorted list of division, date, and consideration_score records, highest urgency first.
- Outputs stored in the ConsiderationScore database table for consumption by the resource allocation agent.

Resource Allocation Agent

- Implemented in app/services/resource_allocation_agent.py; main entry point is run_allocation_agent(db, admin_decisions, target_date).
- Reads division resource inventory from a synthetic CSV dataset including hospital bed capacity, emergency shelters, ambulance count, food stock in tons, clean water capacity in litres, and power grid resilience.
- Queries live DisasterRisk and ConsiderationScore records from the database to build a current hazard and priority context for all divisions.
- Constructs a structured system prompt and sends context to the Gemini 2.0 Flash model via the google-genai client library.
- Returns a structured AllocationResponse containing the allocation plan text, generation timestamp, count of divisions analysed, and a list of high-risk divisions.

- Gemini model and API key are configured via GEMINI_MODEL and GEMINI_API_KEY environment variables in app/core/config.py.

AI Prediction Pipeline

- Implemented as a standalone offline training and validation workspace separate from the runtime FastAPI service.
- Base learners: XGBoost and LightGBM trained on historical climate and hazard data.
- Ensemble method: soft-voting over base model probability outputs.
- Temporal splitting applied to training and test splits to preserve time-series structure and prevent data leakage.
- Hazards covered: Flood, Landslide, Drought.
- Outputs: probabilistic severity predictions per division, exported to CSV and loaded into the operational database.

Kafka Event Publisher

- J2 publishes all processed data and prediction results to Kafka topics after persisting them to PostgreSQL.
- Other subgroups (J3, J4) consume from these topics; J2 does not depend on any downstream consumer.
- Topics are organised by hazard type and data category to allow consumer groups to subscribe selectively.

Docker Configuration

- Dockerfile builds the container, installs dependencies from requirements.txt (including google-genai for Gemini integration), runs Alembic migrations, and starts Uvicorn.
- Integrated with the root docker-compose.yml which provides the shared PostgreSQL service.
- Docker Compose environment variables inject DATABASE_URL and GEMINI_API_KEY at runtime.

2.1.3 J3 Components:

1. Web Application

The primary user interface built with Next.js (React) and styled with Tailwind CSS.

Frontend Framework: Next.js utilizing the App Router for fast, server-side rendered pages and optimized routing.

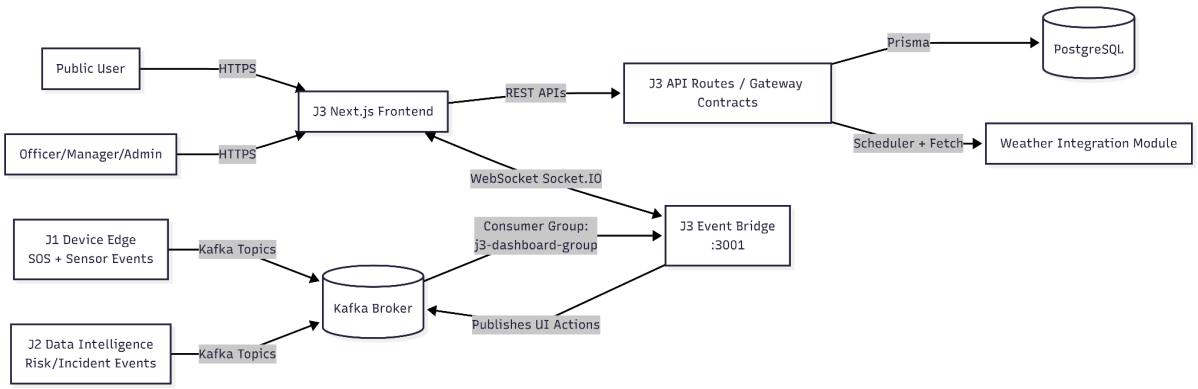
Geospatial Mapping: Integrates Mapbox GL (via react-map-gl and maplibre-gl) to render real-time interactive incident maps, showing disaster zones and active resources.

Real-time Communication: Uses Socket.IO client connections to listen for broadcasted disaster events from the backend, instantly updating the UI without requiring page refreshes.

State Management: Utilizes React Context and custom hooks to manage complex local states like active filters, selected incidents, and user authentication tokens.

Component Architecture

The system utilizes a distributed event-driven architecture connecting edge devices, intelligence engines, and the command dashboard via a central message broker.



Websocket communication flow

Data Source (Kafka): Acts as the primary message broker for system-wide telemetry and reports.
Event Bridge (`event-bridge.js`): A dedicated Node.js service that subscribes to Kafka topics and translates them into WebSocket events.
Frontend Interface (React/Next.js): Consumes events via a global `SocketProvider` and `SocketContext` to trigger immediate UI state updates.

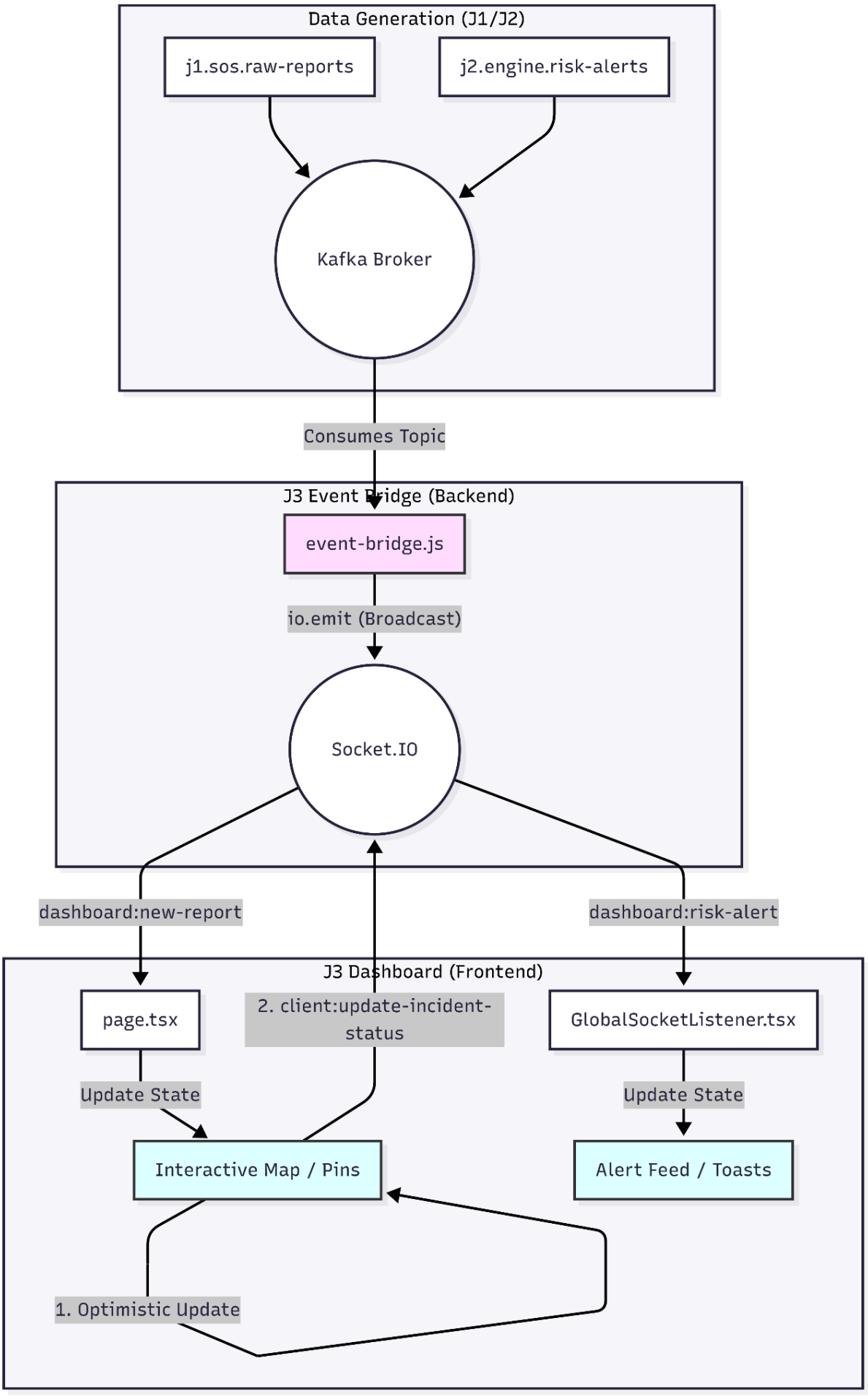
Direction	Event Name	Description

Server → Client	dashboard:new-report	Real-time delivery of incoming SOS and field reports.
Server → Client	dashboard:risk-alert	Critical safety and risk-level updates for the dashboard.
Client → Server	client:update-resource-status	UI-driven updates to resource availability, pushed back to Kafka.

Real-time Event Lifecycle & UI Synchronization

The J3 system employs a unidirectional event-driven pipeline to ensure the dashboard reflects the latest disaster data with sub-second latency.

- **Upstream Production:** High-priority events are generated by external microservices (e.g., `j1.sos.raw-reports` for incoming distress signals or `j2.engine.risk-alerts` for AI-driven predictions) and published to the central Kafka broker.
- **Bridge Consumption:** The J3 **Event Bridge** (`event-bridge.js`) acts as a dedicated consumer, listening to these Kafka topics and immediately translating them into WebSocket emissions.
- **Global Broadcast:** Using `io.emit`, the bridge broadcasts updates to all connected dashboard clients simultaneously. This ensures a synchronized "common operating picture" across all command center screens without the overhead of room-based management.
- **Frontend Reception:** * `GlobalSocketListener.tsx` monitors for `dashboard:risk-alert` and `dashboard:new-report` to trigger system-wide notifications and toasts.
    - `page.tsx` specifically listens for `dashboard:new-report` to dynamically update the geospatial map.
- **State Reconciliation & Rendering:** Upon receiving an event, the UI utilizes React's `setState` to append the new data to the local state. This triggers an immediate rerender of map markers, pins, and alert lists.
- **Optimistic UI Actions:** For status changes initiated by the user (e.g., updating an incident status), the dashboard performs an **optimistic update**. The UI reflects the change locally first to ensure zero perceived lag, then emits a `client:update-incident-status` event back to the bridge for future backend synchronization.


Data Generation (J1/J2)
j1.sos.raw-reports
j2.engine.risk-alerts
Kafka Broker
Consumes Topic
J3 Event Bridge (Backend)
event-bridge.js
io.emit (Broadcast)
Socket.IO
dashboard:new-report
dashboard:risk-alert
J3 Dashboard (Frontend)
page.tsx
2. client:update-incident-status
GlobalSocketListener.tsx
Update State
Update State
Interactive Map / Pins
Alert Feed / Toasts
1. Optimistic Update

2. Mobile Application

Mobile Architecture Diagram

The following describes the architecture of the J3 Field Officer Mobile App, showing how the field officer interacts with the app screens, services, and state management components.

App Type	Flutter cross-platform mobile application (Android & iOS)
Architectural Pattern	Screen-based navigation with dedicated services for auth and incident management
Session Handling	In-memory session managed by AuthService; stores the current officer profile after sign-in
State Updates	Broadcast update stream from IncidentService propagates changes across screens in real time
Navigation	Flutter navigation with bottom navigation bar and drawer for movement across main screens

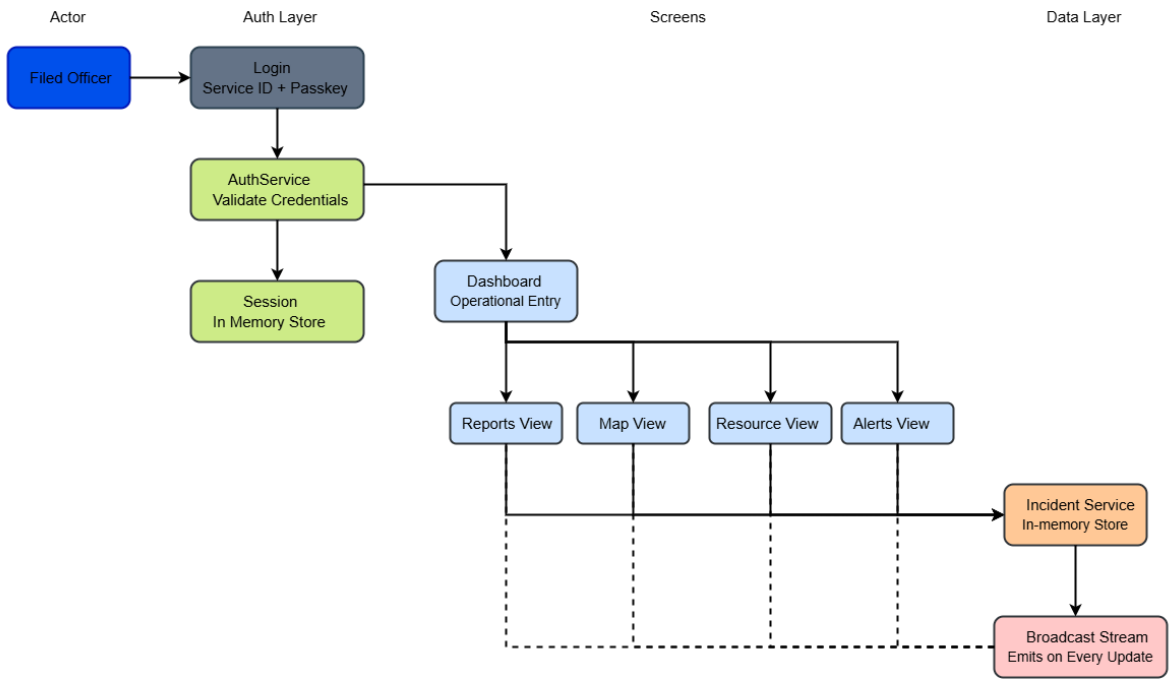
Component Overview

The diagram below maps the relationships between the field officer, screens, services, and the update stream:

Actor / Entry Point	Screens	Services & State
Field Officer	Login Screen Dashboard Screen Reports Screen Map Screen Resources Screen Alerts Screen	AuthService IncidentService In-memory session Broadcast update stream

Architecture Flow

The diagram describes the full component flow:



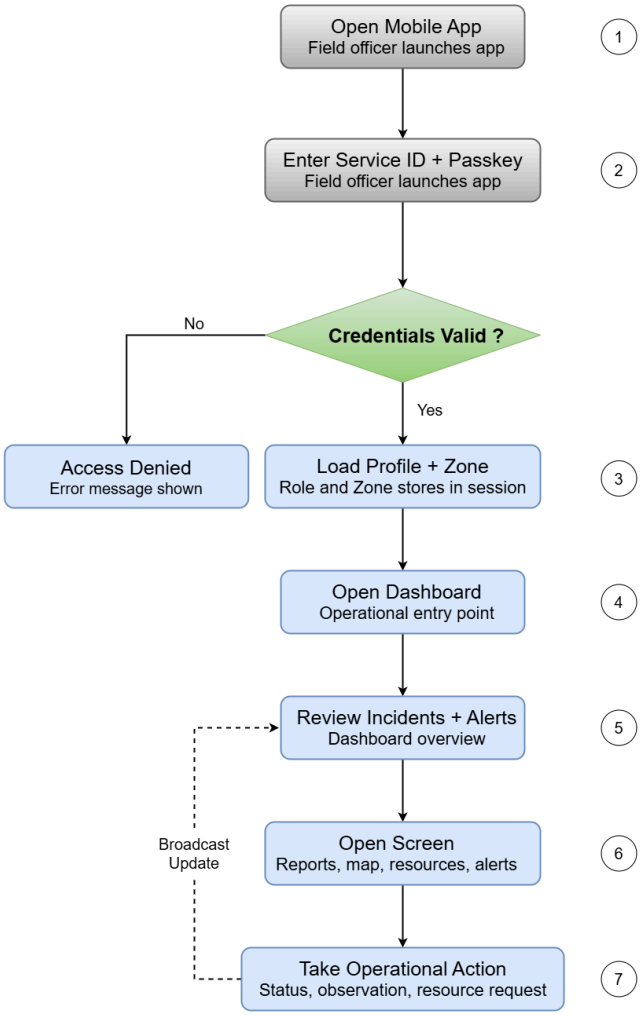
Field Officer Workflow Diagram

The following workflow diagram describes the step-by-step journey of a field officer from app launch through to completing an operational action.

Step	Action	Description
1	**Open mobile app**	Field officer launches the J3 mobile application on their device.
2	**Enter Service ID and passkey**	Officer enters their assigned service ID and passkey on the Login screen.
3	**Credential validation**	AuthService checks the credentials against the local credential map. If invalid, an access denied message is shown. If valid, the flow continues.
4	**Load officer profile and zone**	AuthService loads the officer profile, including their assigned role and operational zone, and stores it in the in-memory session.
5	**Open dashboard**	The officer is navigated to the Dashboard screen, which serves as the operational entry point for all subsequent actions.
6	**Review incidents and alerts**	The officer reviews active incidents, alert summaries, and operational metrics displayed on the dashboard.
7	**Open reports, map, resources, or alerts**	Using the bottom navigation bar or drawer, the officer navigates to the screen most relevant to their current task.
8	**Take operational action**	The officer performs a context-appropriate action: updating a status, recording an observation, or requesting resources.

9	**Broadcast update to app state**	IncidentService processes the action and emits an update through the broadcast stream. Relevant screens react to the change automatically.

Workflow Diagram Source



Mobile Workflow Notes

Screen Responsibilities

- Login flow uses the service ID and passkey pair defined in the local auth service.
- Dashboard acts as the operational entry point after sign-in.
- Reports screen filters incidents by the signed-in officer's zone.
- Map screen presents incident markers and action sheets for field decisions.
- Resources screen shows operational assets and lets the user inspect and filter them.
- Alerts screen summarizes active alerts and system status for quick triage

2.1.4 J4 Components: [Describe API Gateway routing, Keycloak service.]

2.3 Specific Design Considerations (Small Scale)

- **API Contracts:** [J2 & J3]

API Contracts (J2 and J3)

The J2 REST API exposes the following contracts consumed by J3 and other subgroups:

- GET /health returns { "status": "ok" }. Used by J4 infrastructure monitoring to verify service liveness.
- GET /api/v1/intelligence returns { "service": "j2-data-intelligence", "status": "ready" }. Confirms the service is operational.
- POST /api/v1/intelligence/agent/allocate accepts a JSON body with admin_decisions (string) and an optional target_date (date). Returns allocation_plan, generated_at timestamp, divisions_analyzed count, and high_risk_divisions list. Returns 503 when the Gemini API is unavailable and 502 when it returns an error.
- All endpoints require a valid Bearer token issued by J4 Keycloak. Requests without a valid token receive a 401 Unauthorised response.
- Request and response payloads are validated against Pydantic schemas defined in app/schemas/. Invalid payloads return a 422 Unprocessable Entity response with field-level error details.

Kafka Event and Message Schemas (J2 and J4)

J2 publishes to the following Kafka topic categories after data is persisted to PostgreSQL:

- Environmental readings topic carries IoT device measurement events including device ID, division ID, measurement type, value, unit, and timestamp.
- Flood severity topic carries flood severity events per division including division ID, severity level, contributing metric values, and timestamp.
- Hazard prediction topics (one per hazard type: flood, landslide, drought) carry probabilistic severity prediction events including division ID, hazard type, predicted severity class, class probability scores, and prediction timestamp.
- Consideration score topic carries the population-weighted priority score per division including division ID, score value, dominant hazard class, severity multiplier applied, and timestamp.

All Kafka messages are serialised as JSON. J3 and J4 consumer groups subscribe to the relevant topics and are responsible for handling schema evolution through backward-compatible consumer logic.

Consideration Score Design

- The score is computed as a normalised value between 0 and 1, where higher values indicate greater urgency.
- The dominant hazard class is identified by the highest class probability across Normal, Moderate, Severe, and Extreme.
- The severity multiplier for the dominant class is multiplied with the normalised population value and passed through a logistic function with scaling constant k = 10.0 by default.
- Flat population ranges default to a normalised population of 1.0 to ensure hazard severity still drives the score in the absence of population differentiation.
- Scores are stored to four decimal places and sorted descending so the most urgent divisions appear first in all API responses and agent inputs.

Database Schema Design

- All tables are created and managed exclusively through Alembic migration scripts; no manual DDL is applied.
- Foreign key relationships link IoTDeviceData to IoTDevice, and all risk and response entities to the relevant Division record.
- Timestamp columns use UTC throughout to avoid timezone inconsistencies across the distributed system.
- The ConsiderationScore table stores the computed priority score alongside the contributing hazard probability values and severity multiplier for full auditability.

Configuration and Environment Management

- All sensitive configuration (database credentials, Gemini API key, service ports) is supplied via environment variables read from a .env file locally or injected by Docker Compose in containerised deployments.
- app/core/config.py centralises configuration parsing using Pydantic settings management, exposing APP_NAME, APP_VERSION, APP_HOST, APP_PORT, DATABASE_URL, GEMINI_API_KEY, and GEMINI_MODEL.
- Local .env uses localhost as the database host; Docker deployments use the postgres compose service name.

Machine Learning Artefact Versioning

- Trained model files are stored in Models/ and must be versioned alongside the prediction CSVs in Predictions/ and the Model_Training_Report.md.
- If the training pipeline is updated, the Consideration_Score_Methodology.md, prediction outputs, and the report must all be regenerated and committed

together to maintain reproducibility.

- The prediction pipeline workspace is fully separate from the runtime FastAPI service and introduces no runtime dependencies into it.

2.3.1 Internal Dashboard Routes for Next.js (J3)

The following endpoints are implemented within the J3 repository as Next.js route handlers. Currently, these return structured JSON backed by mock data for frontend development and testing.

Method	Route	Description / Response Payload
POST	/api/auth/login	Authenticates users. Returns { success, user, token }.
GET	/api/dashboard/overview	Dashboard summary: { activeIncidents, criticalAlerts, peopleAffected, resources, alerts[] }.
GET	/api/incidents	List of Incident[] including severity, location, and population data.
GET	/api/resources/list	Deployment data: { resourcesList[], deploymentStats }.
GET	/api/relief/shelter	Relief status: { activeShelters, totalOccupancy, stockLevels }.
GET	/api/analytics/situation	Performance metrics: { avgResponseTime,

		totalRelief, totalRescued }.
GET	/api/activity	Returns chronological activity log entries for the audit trail.

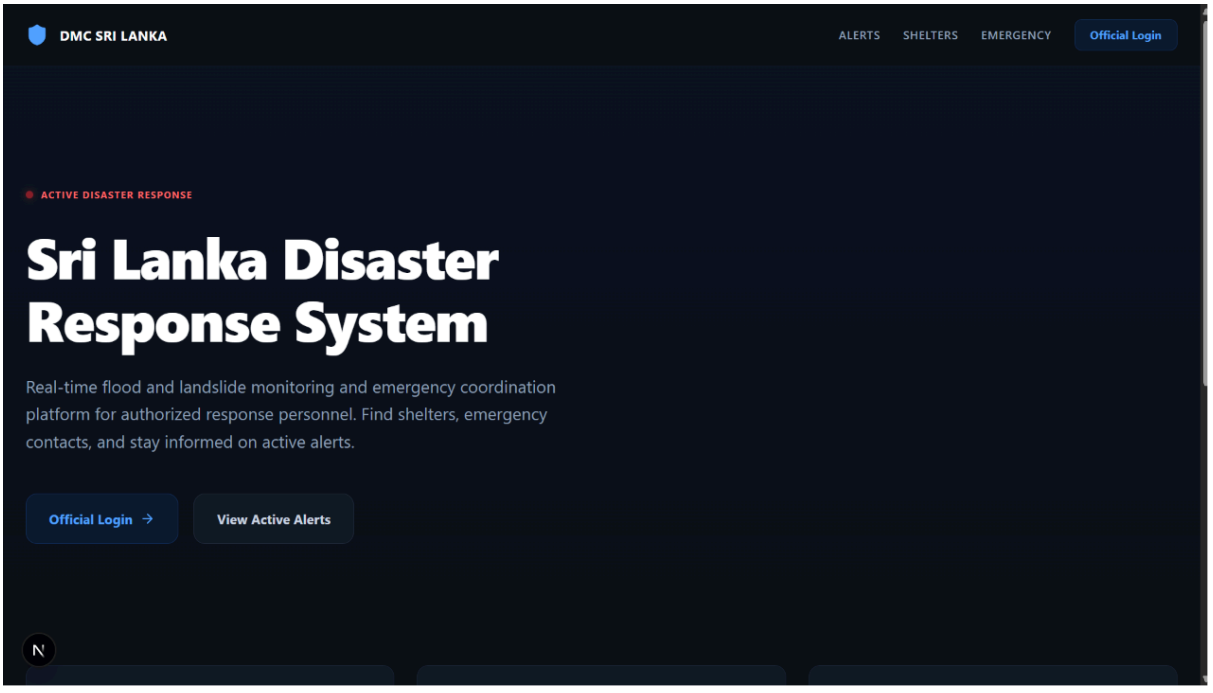
- **Event/Message Schemas:** [J2 & J4 - Kafka topics]

2.3.2 UI/UX Wireframes: [J3]

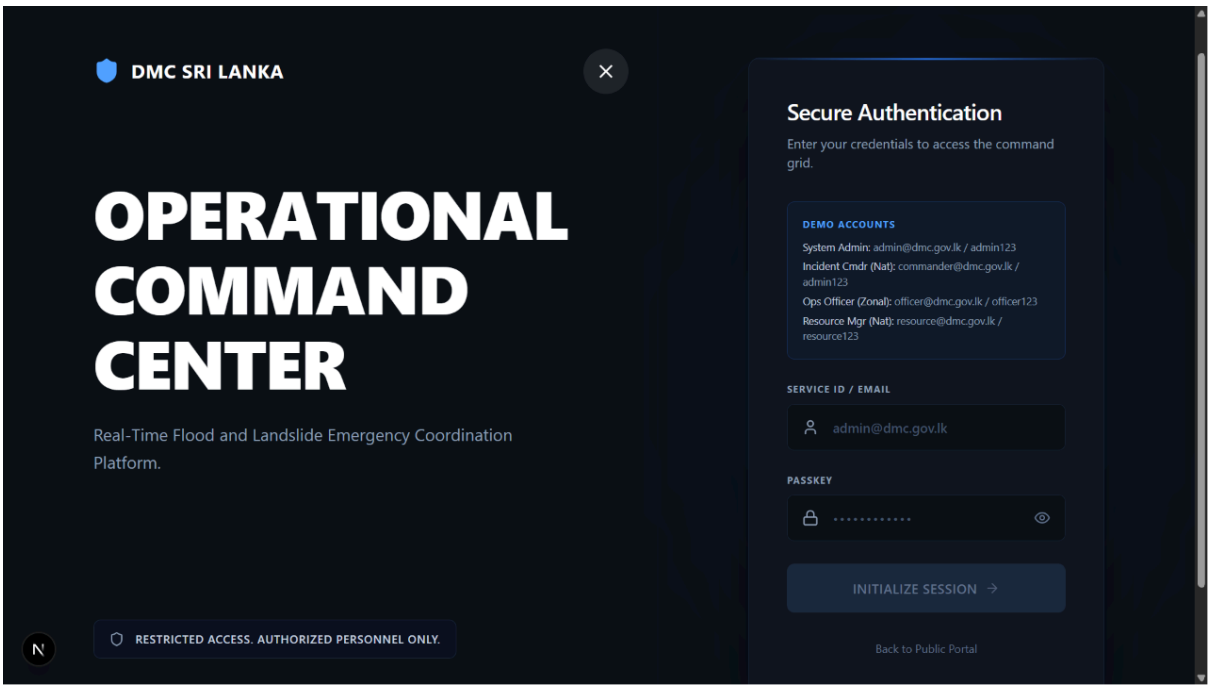
Command Dashboard

1. Screen & Page Designs

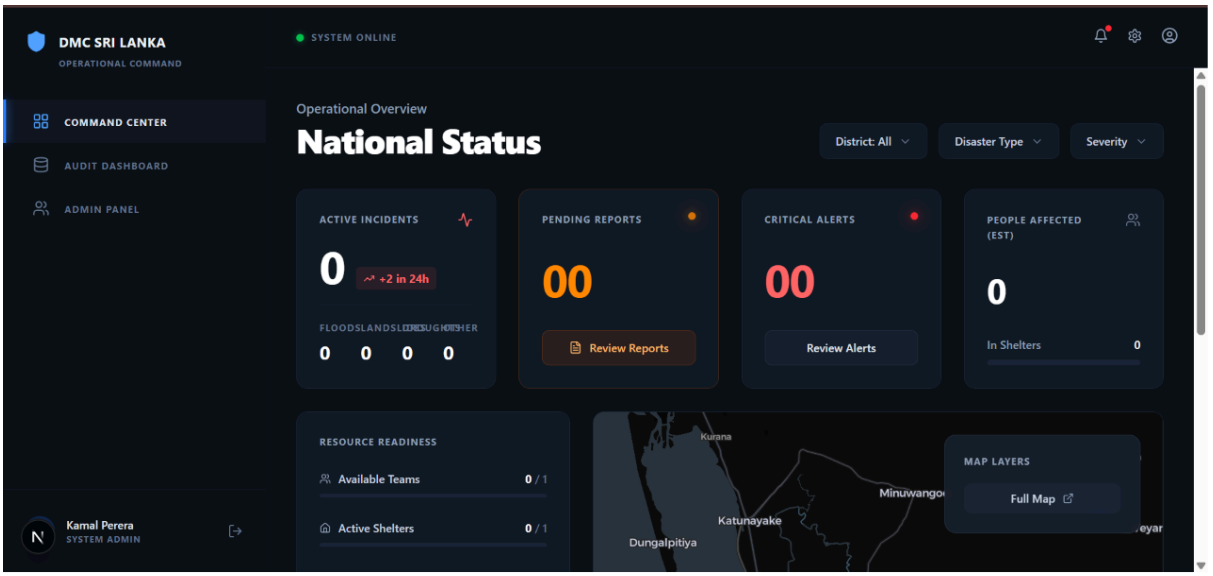
Landing page:



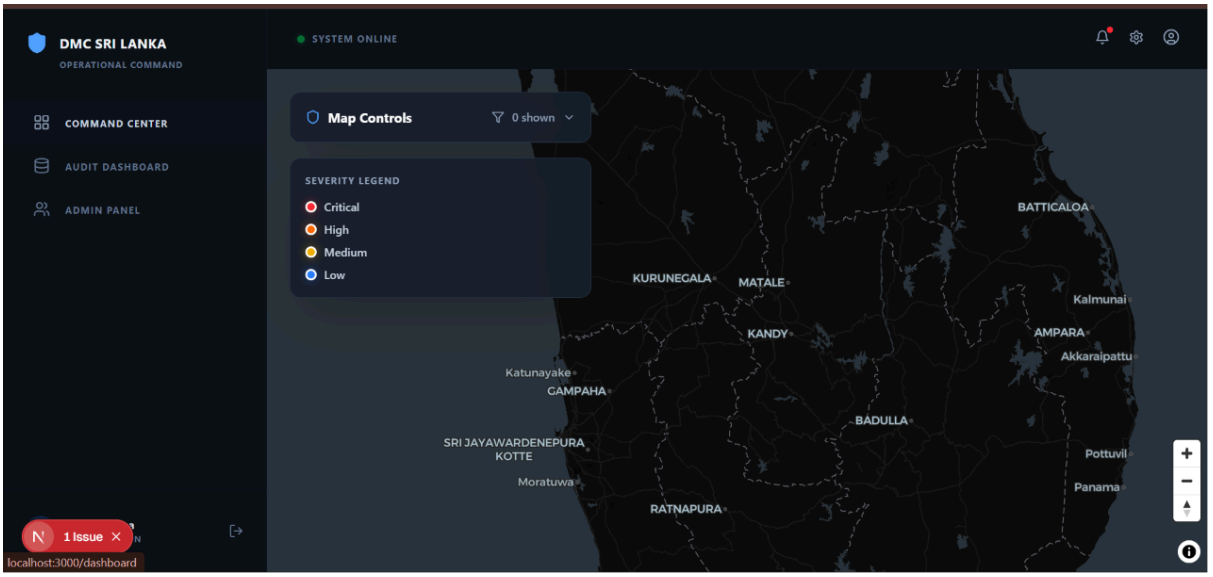
Authentication Portal: A secure login screen interfacing with the J4 Keycloak provider.



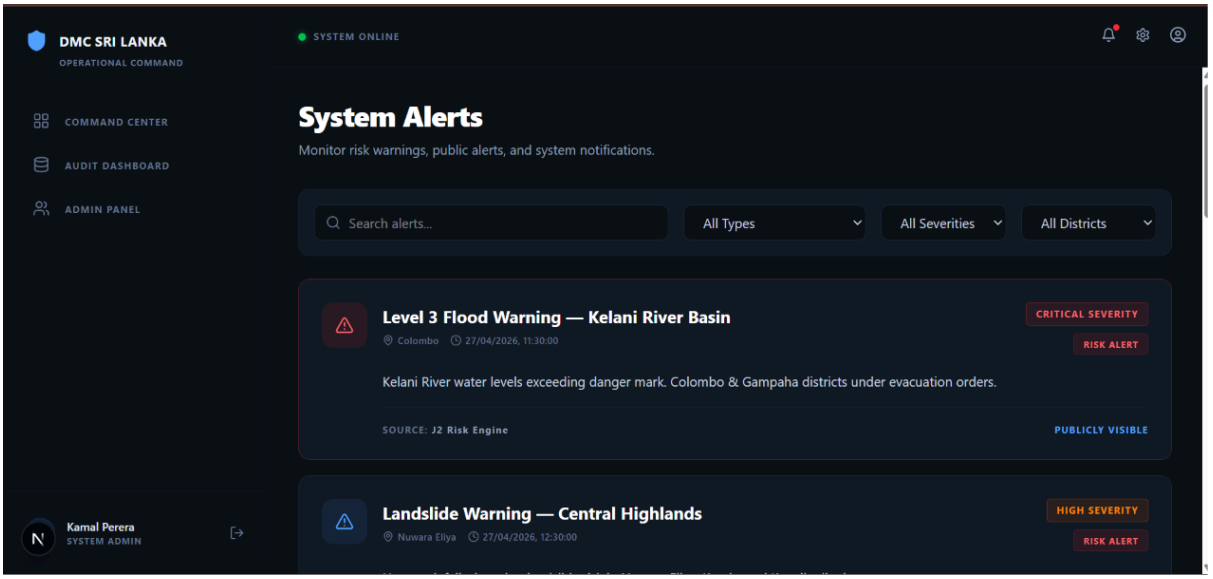
Overview Dashboard: The landing page featuring high-level statistics (Total Incidents, Active Resources) using Recharts for data visualization.



Interactive Incident Map: A full-screen or split-screen map view displaying color-coded markers (Red = Critical, Yellow = High) for active disasters. Includes clustering for high-density events.



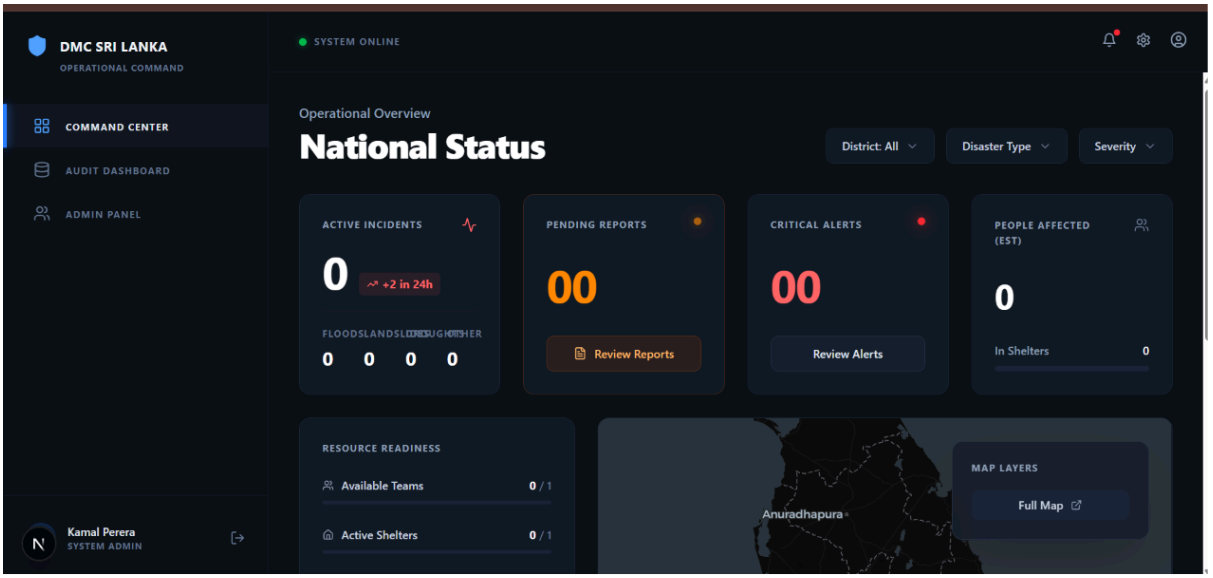
Alerts: A live feed of incoming disaster alerts



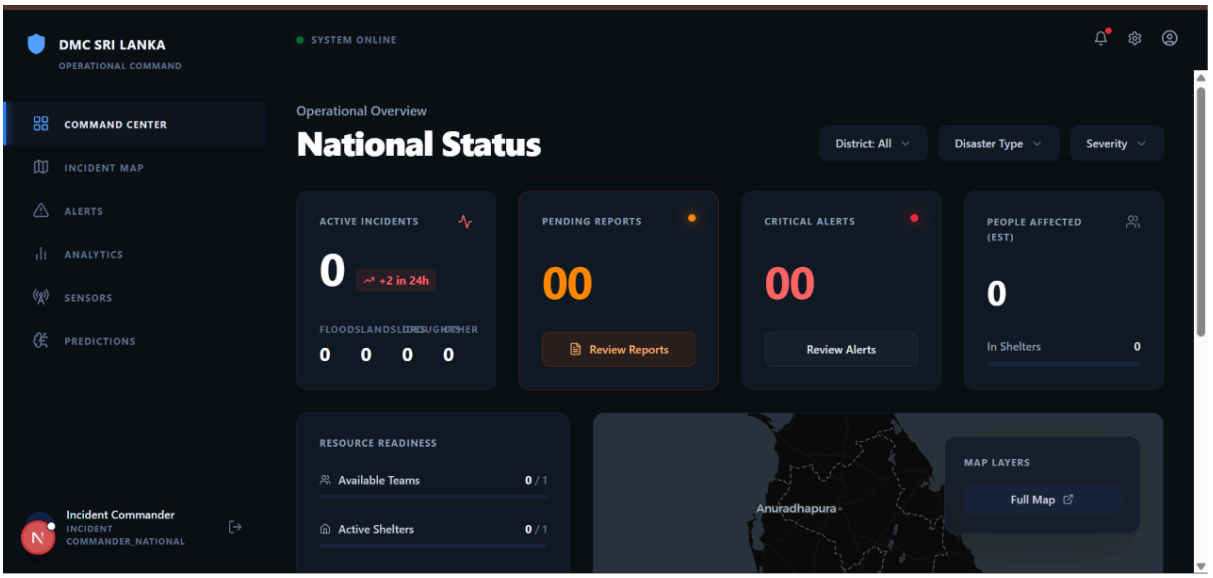
2. Role-Based UI Layouts

The UI dynamically adapts based on the authenticated user's role (RBAC):

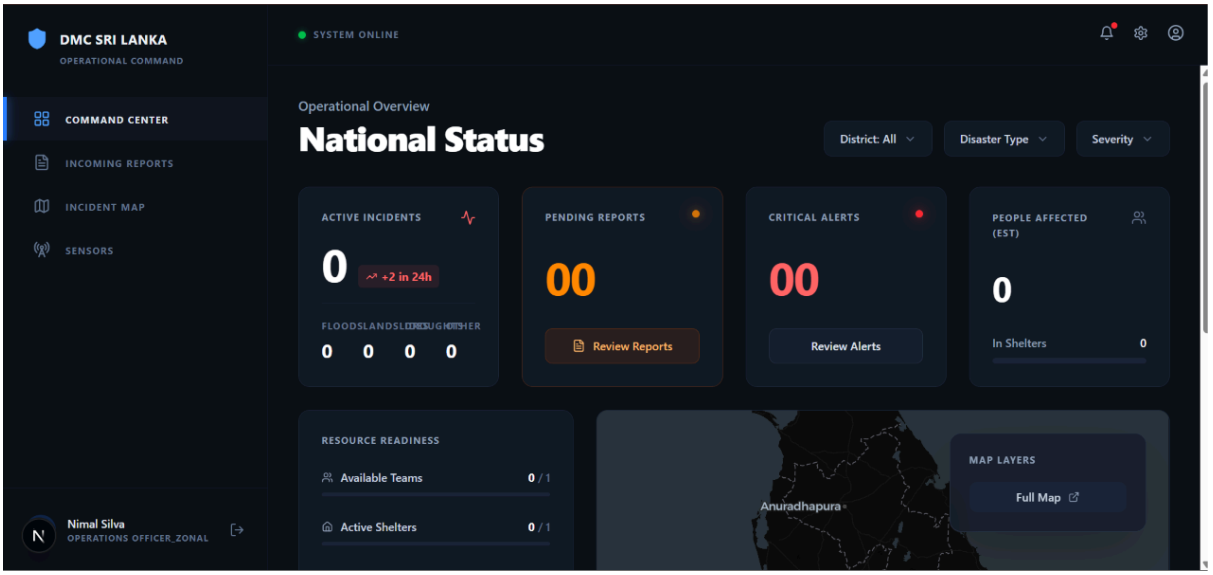
System Admin: Has access to all views, including backend system health metrics and user management.



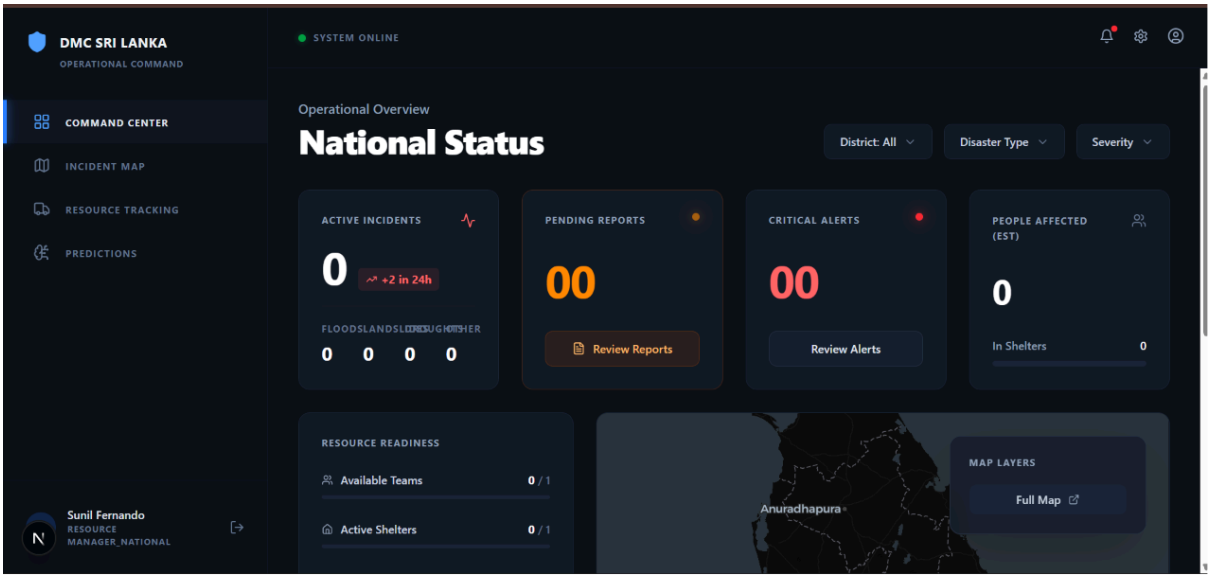
Commander (Command Center): Focuses on the macro view. Dashboard displays overarching statistics, the interactive map with full dispatch capabilities, and resource management tools to assign units to incidents.



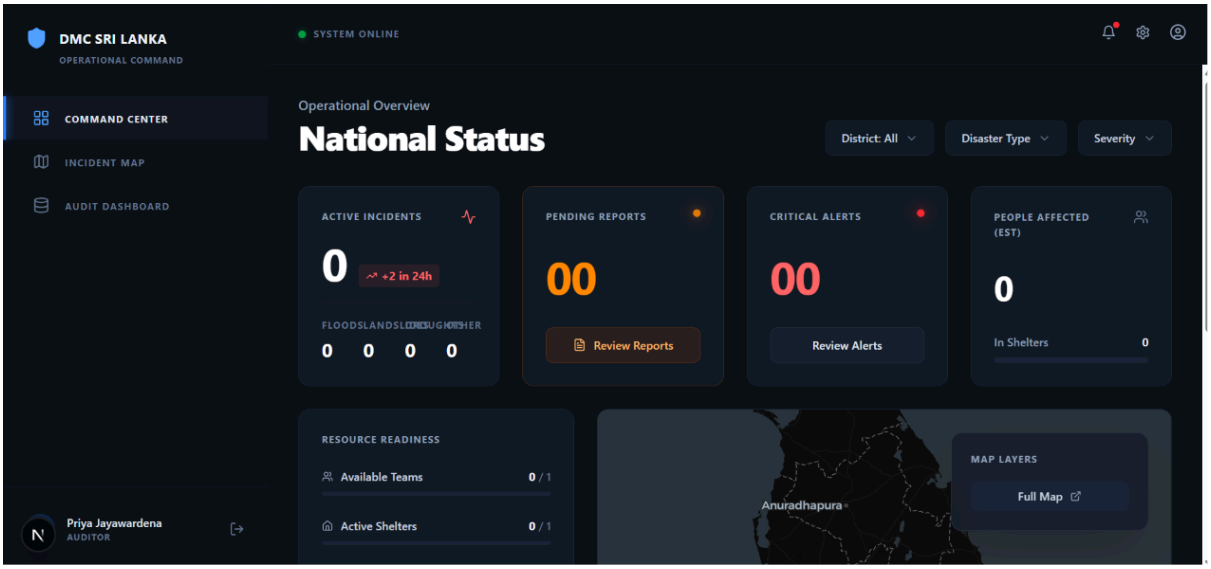
Operations Officer: Focuses on the micro view. The UI simplifies to prioritize their specific assigned tasks, turn-by-turn navigation on the map, and quick-action buttons to update incident statuses (e.g., "En Route", "On Scene", "Resolved").



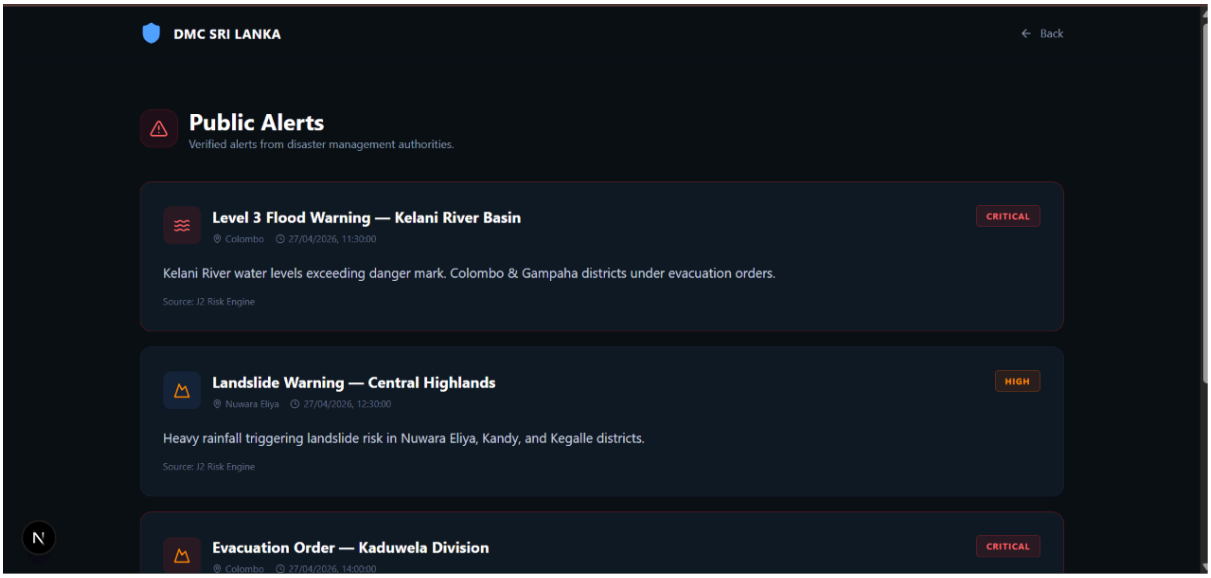
Resource Manager: Responsible for dispatching resources



Auditor: Can view audit reports



Public portal: Public can view alerts, shelters and emergency contacts without logging to the the system



2.3.3 Mobile App Wireframes and User Interaction Designs

The Field Officer App is a cross-platform mobile application serving as the mobile extension of the Command Dashboard. It is designed to provide real-time location-based updates, improve situational awareness, and allow users to report incidents and conditions.

Screen Specifications

Based on the defined user interaction requirements, the mobile application consists of the following core screens:

- **Command Secure Access Portal (Login):** Ensures only authorized personnel can

access the system. Contains input fields for Designation, Officer name, Service ID, and a Secure Passkey, along with an "Initialize Session" action button.

- **Operations Live (Dashboard):** Provides an immediate overview of the active operational area. Displays the count of Critical Alerts and Active Incidents. Shows a "Resource Readiness" percentage and a "National Status" section.
- **Field Reports:** Allows officers to manage and verify incidents assigned to them. Contains filterable tabs for "All", "Assigned", and "Priority". Displays specific incident cards with actionable "Verify" and "Reject" buttons.
- **Matrix (Incident Map):** Provides geographical context for active incidents and resources. An interactive map plotting incidents by severity and identifying shelter locations. Includes an "Active AO Summary" showing personnel counts.
- **Capacity Overview (Resources):** Tracks available physical assets and shelter capacity. Displays shelter metrics (e.g., North Zone Shelter) and lists active units with their transit statuses.
- **Active Alerts:** Delivers push notifications and system-wide broadcast messages. Shows the overall System Health and lists critical alerts with detailed descriptions.

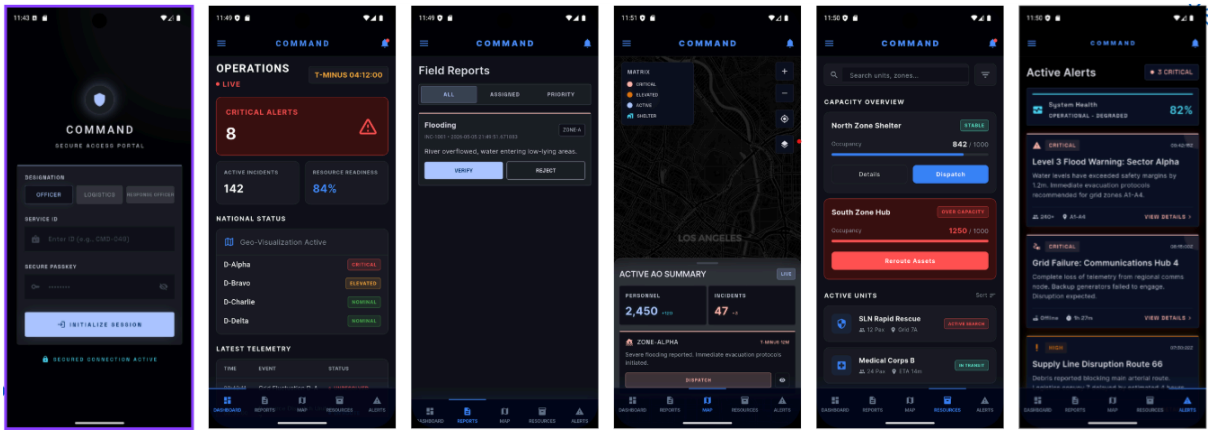


Figure 2: Field Officer Mobile Application Interface. From left to right: Secure Login Portal, Live Operations Dashboard, Field Reports Queue, Active AO Matrix Map, Resource Capacity Overview, and Critical Alert Feed.

Mobile Navigation and Interaction Flow The mobile app relies on a resilient, offline-first architecture designed for disaster conditions where connectivity may be unstable.

- **Offline-First Interaction:** The app captures disaster data offline, storing it in a local SQLite database. It utilizes an offline buffering mechanism where data is stored locally when Wi-Fi or cellular networks are unavailable.
- **Synchronization Flow:** The app features an automatic sync and retry mechanism with exponential backoff. It automatically transmits buffered data once a connection is re-established, surviving network failures without data loss or duplication. Exact delivery is ensured using a UUID and Idempotency-Key system.

- **Security Architecture:** [J4 - Auth flows, Role-Based Access]

- **Monitoring and Observability Architecture**

The observability system sits across four layers. Services in J1-J4 are instrumented to expose metrics at '/metrics' endpoints and emit structured logs to stdout. A collection layer ingests this telemetry: Prometheus pulls metrics every 15 seconds via HTTP, while Filebeat captures container logs and pushes them through Logstash into Elasticsearch. An alerting layer evaluates Prometheus rules and routes firing alerts through Alertmanager. A visualisation layer Grafana queries both Prometheus and Elasticsearch to present dashboards and SLO compliance views.

Two independent pipelines operate in parallel.

The **metrics pipeline** is pull-based and deterministic: Prometheus scrapes, stores, and evaluates on a fixed 15-second cadence.

The **log pipeline** is push-based and asynchronous: Filebeat buffers up to 100 events and flushes within 5 seconds to Logstash, which parses and indexes into daily Elasticsearch indices (drs-logs-YYYY.MM.dd).

See Figure 1 (data flow) and Figure 2 (alert lifecycle) in the accompanying diagrams.